Complete Python Notes

@Tajamulkhann
@Tajamul.Codes

Introduction to Python

① What is Python?

Python is a high-level, interpreted, general-purpose programming language. It emphasises code readability with simple and clean syntax. Python is versatile, powerful and designed to be easy to learn and use.

Key Point

Python is easy to learn, powerful to use, and widely adopted in industry.

② Features & Advantages

- ✓ **Simple & Readable** – Clean syntax similar to English.
- ✓ **Interpreted** – No compilation step, run code line by line.
- ✓ **High-Level Language** – Focus on logic, not on complex syntax.
- ✓ **Cross-Platform** – Works on Windows, macOS, Linux and more.
- ✓ **Large Standard Library** – Built-in modules for almost everything.
- ✓ **Extensible** – Can be extended using C, C++ and other languages.
- ✓ **Community Support** – Huge & active community worldwide.

③ Applications of Python

- Web Development
- Data Science & Analytics
- Machine Learning
- Automation & Scripting
- DevOps & Infrastructure
- Cybersecurity
- Game Development

④ Python Installation

1. Go to the official Python website: https://www.python.org/downloads/
2. Download the latest Python 3.x version.
3. Run the installer and CHECK "Add Python to PATH".
4. Click "Install Now" and finish setup.
5. Verify installation in terminal / command prompt:

```
$ python --version
Python 3.12.3
```

Pro Tip: Keep Python and pip up to date for better performance and security. ✓

⑤ Running Your First Program

1. Create a file named hello.py
2. Write the following code:

```
1 print("Hello, World!")
2
```

3. Run the program:

```
$ python hello.py
Hello, World!
```

Try it!

☐ Try: print("Hello, World!")

Pro Tip: Python files end with .py extension. Use meaningful file names like hello.py

⑥ Python Execution Flow

1. **Write Code** – Write your Python code in a .py file.
2. **Interpreter** – Interpreter reads the code line by line.
3. **Execute** – Interpreter executes each line of code.
4. **Output** – Output is displayed on the screen.

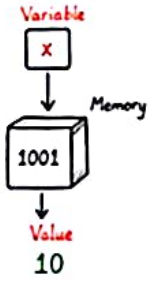
Interview Tip: Python's simplicity, readability and vast ecosystem make it one of the most popular programming languages in the world. ✓

LinkedIn: @Tajamulkhan | Instagram: @Tajamul.Codes

Variables & Data Types

1 Variables in Python

A variable is a name given to a memory location where a value is stored.
In Python, variables are created when you assign a value to them.
Python has no command for declaring a variable.




```
# Example
name =          # String
age  = 25       # Integer
price = 19.99   # Float
is_active = True  # Boolean
```

2 Variables Naming Rules

- ✓ Must start with a letter (a-z, A-Z) or underscore (_).
- ✓ Can contain letters, digits (0-9) and underscore (_).
- ✓ Case sensitive (myVar, myvar are different).
- ✓ Cannot start with a digit.
- ✓ Cannot use Python keywords.
- ✗ No spaces or special characters (!, @, #, $, %, etc.)

✓ Examples	✗ Invalid
• my_var	• 2value
• userName	• user-name
• _count	• my var
• total2	• class

3 Common Data Types

Data Type	Description	Example	Type
Integer (int)	Whole numbers (positive or negative)	10, -25, 0	int
Float (float)	Decimal numbers	3.14, -0.001, 2.0	float
String (str)	Sequence of characters enclosed in quotes	"Hello", "Python"	str
Boolean (bool)	Represents True or False	True, False	bool
List (list)	Ordered, mutable collection	[1, 2, 3], ['a', 'b']	list
Tuple (tuple)	Ordered, immutable collection	(1, 2, 3), ('a', 'b')	tuple
Set (set)	Unordered collection of unique items	{1, 2, 3}	set
Dictionary (dict)	Key-value pairs	{"name": "QA", "age": 25}	dict

4 Numbers

Python supports integers and floating point numbers.

```
a = 10      # int
b = -7      # int
c = 3.1415  # float
d = -0.25   # float
```

☆ Note: Python has no separate type for long integers, it can handle very large numbers.

5 Strings

Strings are sequences of characters enclosed in single, double or triple quotes.

```
s1 = "Hello World"
s2 = "Python is awesome"
s3 = """Multi-line
      string"""
```

Tip: Strings are immutable, meaning they cannot be changed after creation

6 Boolean

Boolean type has two values: True or False.

```
is_valid  = True
is_admin  = False
```

☆ Note: True and False must start with a capital letter.

7 Type Conversion

Convert one data type to another.

```
int('10')      # str -> int
float('3.14')  # int -> float
str(100)       # int -> str
bool(0)        # int -> bool (False)
bool(1)        # int -> bool (True)
```

☆ Interview Fact: Type conversion is explicit in Python No automatic implicit conversion.

8 Type Checking

Use type() function to check the data type.

```
x = 25
y = "Python"
z = 3.14
print(type(x))   # <class 'int'>
print(type(y))   # <class 'str'>
print(type(z))   # <class 'float'>
```

Pro Tip: Understanding data types is the key to writing efficient and bug-free Python code. ✓

Input, Output & Operators

1 input() Function

input() is used to take input from the user.
It always returns data in String type

```
name = input("Enter your name: ")
age  = input("Enter your age: ")
print("Hello", name, "!")
```

Example Output:

```
Enter your name:
Enter [illegible]
Hello
```

2 print() Function

print() is used to display output on the screen.
Supports multiple values, sep, end and formatting

```
print("Python is awesome!")
print("A", "B", "C", sep="-", end="\n")
x = 10
print(f"Value of x is {x}")
```

Example Output:

```
Python is awesome!
A-B-C
Value of x is 10
```

Pro Tip
Use f-strings for clean and readable output formatting.

3 Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 3	1.6667
//	Floor Division	5 // 3	1
%	Modulus	5 % 3	2
**	Exponentiation	5 ** 3	125

4 Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	5 <= 3	False

5 Logical Operators

Operator	Meaning	Example	Result
and	Logical AND	(5 > 3) and (2 < 4)	True
or	Logical OR	(5 > 3) or (2 > 4)	True
not	Logical NOT	not(5 > 3)	False

Note: Logical operators return boolean values (True / False).

6 Assignment Operators

Operator	Example	Same As
=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
//=	x //= 5	x = x // 5
%=	x %= 5	x = x % 5
**=	x **= 5	x = x ** 5

Tip:
Use compound assignment operators to make your code short & efficient

7 Membership Operators

Operator	Meaning	Example	Result
in	True if value exists	"a" in "apple"	True
not in	True if value does not exist	"x" not in "apple"	True

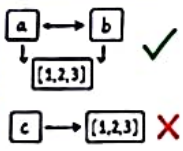
Example →

```
lst = [1, 2, 3, 4]
2 in lst      # True
5 not in lst  # True
```

8 Identity Operators

Operator	Meaning	Example	Result
is	True if both references point to same object	a is b	True or False
is not	True if both references point to different objects	a is not b	True or False

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)  # True
print(a is c)  # False
```



Real-world Use Cases

- Taking user input in CLI applications
- Performing calculations in automation scripts
- Validating conditions and decisions in test scripts
- Checking data presence in collections
- Comparing objects and references in frameworks

Interview Tip

- ✓ Know the difference between "==" and "is".
- ✓ Remember operator precedence: Arithmetic > Comparison > Logical > Assignment
- ✓ Practice with small examples to build strong fundamentals.

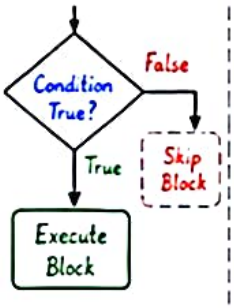
Decision Making in Python

python

① if Statement

Executes a block of code if the condition is True.

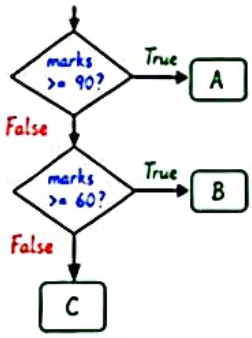
```
age = 18

if age >= 18:
    print("You are eligible
          to vote.")
```



② if-elif-else Ladder

Checks multiple conditions in sequence.

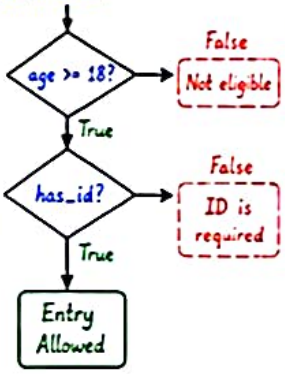
```
marks = 75
if marks >= 90:
    grade = 'A'
elif marks >= 60:
    grade = 'B'
else:
    grade = 'C'
print("Grade:", grade)
```



③ Nested if Conditions

An if or else block inside another if block.

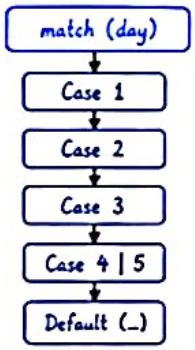
```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry Allowed")
    else:
        print("ID is required")
else:
    print("Not eligible")
```



④ match-case (Python 3.10+)

Selects a block of code based on the value of an expression.

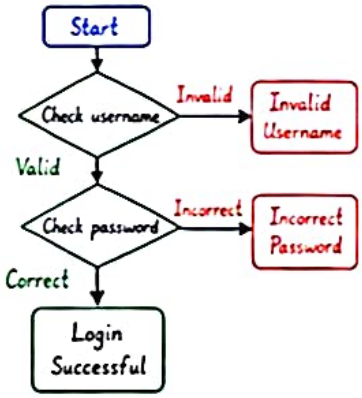
```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4 | 5:
        print("Midweek")
    case _:
        print("Weekend")
```



⑤ Real-world Example – Login System

Validates username and password with multiple conditions.

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin":
    if password == "1234":
        print("Login Successful ✓")
    else:
        print("Incorrect Password ✗")
else:
    print("Invalid Username ✗")
```



Pro Tips

- ✓ Use meaningful conditions.
- ✓ Keep conditions simple and readable.
- ✓ Order conditions from most specific to general.
- ✓ Use match-case for clean and elegant code when matching fixed values.

Interview Fact

Python's decision making statements help control program flow based on different conditions and are widely used in automation scripts, validations, and real-world applications.

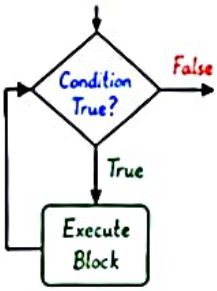
Loops in Python

python

① while Loop

Repeats a block of code while a condition is True.

```
i = 1
while i <= 5:
    print(f"Count: {i}")
    i += 1
```



Example Output:

```
Count : 1
Count : 2
Count : 3
Count : 4
Count : 5
```

② range() Function

Generates a sequence of numbers.
Syntax: range(start, stop, step)

Example	Output
range(5)	0, 1, 2, 3, 4
range(1, 5)	1, 2, 3, 4
range(2, 10, 2)	2, 4, 6, 8

Example in Loop:

```
for i in range(1, 6):
    print(i, end=' ')
```

Output: 1 2 3 4 5

Tip
range() is memory efficient and commonly used in loops.

③ break Statement

Terminates the loop completely.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

Loop stops when break is encountered.

Output:

```
1
2
```

④ continue Statement

Skips the current iteration and continues with the next.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Skips 3 and continues with the next value.

Output:

```
1 2 4 5
```

⑤ pass Statement

Does nothing.
Used as a placeholder.

```
for i in range(1, 4):
    if i == 2:
        pass
    print(i)
```

```
pass
```

Placeholder for future code.

Output:

```
1
2
3
```

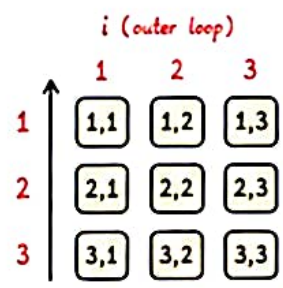
⑥ Nested Loops

A loop inside another loop.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"i={i}, j={j}")
```

Output:

```
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=1
i=3, j=2
i=3, j=3
```



Tip
Nested loops are used in patterns, matrix traversal, and combinational problems.

Real-world Use Cases

- ✓ Iterating through collections of data
- ✓ Reading lines from a file
- ✓ Retry mechanisms in automation scripts
- ✓ Polling APIs until a condition is met
- ✓ Generating reports, tables, or patterns

Interview Fact

Loops are essential in programming for repetition. Choose the right control statement (break, continue, pass) to optimize code flow and performance.

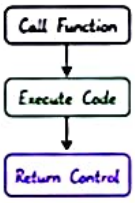
Functions in Python

① Creating Functions

A function is a block of reusable code that performs a specific task.

```
def greet(name):
    """This function greets the user."""
    print(f"Hello, {name}!")
# Function call
```

Flow



Output:

② Parameters & Arguments

Parameters are variables in function definition.
Arguments are values passed while calling.

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)   # 30
```

a, b → Parameters (in definition)

10, 20 → Arguments (in call)

Output: 30

③ Return Values

The return statement sends a result back to the caller.

```
def multiply(x, y):
    return x * y

res = multiply(4, 5)
print(res)   # 20
```

Output: 20

Tip
A function without return returns None by default.

④ Default Arguments

Default values are used when no value is provided in the function call

```
def greet(name, msg="Welcome!"):
    print(f"Hi {name}, {msg}")

greet("Alice")                # Hi Alice, Welcome!
greet("Bob","Good Morning!") # Hi Bob, Good Morning!
```

Output:
Hi Alice, Welcome!
Hi Bob, Good Morning!

Tip
Default args must come after non-default args.

⑤ *args (Arbitrary Positional Arguments)

Allows a function to accept any number of positional arguments.

```
def sum_all(*args):
    total = 0
    for num in args:
        total += num
    return total

print(sum_all(1, 2, 3, 5))  # 15
```

Output: 15

Note
*args is treated as a tuple inside the function.

⑥ **kwargs (Arbitrary Keyword Arguments)

Allows a function to accept any number of keyword arguments.

```
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
print_info(name="Alice", age=25, city="Hyderabad")
```

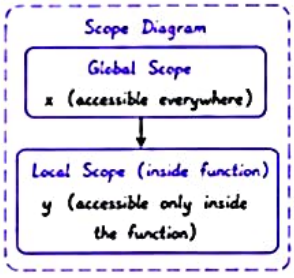
Output:
name: Alice
age: 25
city: Hyderabad

Note
**kwargs is treated as a dictionary inside the function.

⑦ Variable Scope

Scope determines where a variable can be accessed.

```
x = "Global"
def show():
    y = "Local"
    print("Inside function ->", x, y)
show()
print("Outside function ->", x)
# print(y)  # Error! y is not accessible here
```



Key Takeaways

- ✓ Use functions to organize and reuse code.
- ✓ Use parameters to make functions flexible.
- ✓ Return values help get results from functions.
- ✓ *args and **kwargs handle variable inputs.
- ✓ Understand scope to avoid variable conflicts.

Interview Tip

Functions are building blocks of clean, modular and maintainable Python code. Be ready to explain *args, **kwargs and variable scope in interviews! ✓

Strings in Python

① What is a String?

A String is a sequence of characters enclosed in single quotes (`'`), double quotes (`"`) or triple quotes (`"""`). Strings are immutable.

```
s1 = 'Hello'              # Single quoted
s2 = "Python"             # Double quoted
s3 = """Multiline         # Triple quoted
     Line String"""
```

② String Operations

Operation	Example	Result	Description
Concatenation	`'Hello' + ' ' + 'Python'`	`'Hello Python'`	Join strings
Repetition	`'Hi' * 3`	`'HiHiHi'`	Repeat string
Length	`len("Python")`	6	Number of characters
Membership	`'Py' in 'Python'`	True	Check if substring exists

③ Indexing

Access characters using index.
Index starts at 0.

```
s = "Python"

Index:  0  1  2  3  4  5
Char :  P  y  t  h  o  n

s[0]    # 'P'
s[3]    # 'h'
s[-1]   # 'n' (last character)
s[-2]   # 'o'
```

Tip

Negative index starts from -1 (last character).

④ Slicing

Extract a part of the string using start:stop:step.

```
s = "Python Programming"

Index:  0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16
Char :  P  y  t  h  o  n     P  r  o  g  r  a  m  m  i  n  g

s[0:6]      # 'Python'       (start included, stop excluded)
s[7:]       # 'Programming'  (from index 7 to end)
s[:6]       # 'Python'       (from start to index 5)
s[::2]      # 'Pto rgamn g'  (step of 2)
s[::-1]     # 'gnimmargorP'  (negative slicing)
```

Note

Default step is 1.

`s[::-1]` reverses the string.

⑤ Common String Methods

Method	Example	Result	Description
`lower()`	`"PYTHON".lower()`	`"python"`	Convert to lowercase
`upper()`	`"python".upper()`	`"PYTHON"`	Convert to uppercase
`capitalize()`	`"python".capitalize()`	`"Python"`	First char uppercase
`title()`	`"hello world".title()`	`"Hello World"`	Capitalize each word
`strip()`	`"  hi  ".strip()`	`"hi"`	Remove leading/trailing spaces
`lstrip()`	`"  hi  ".lstrip()`	`"hi  "`	Remove leading spaces
`rstrip()`	`"  hi  ".rstrip()`	`"  hi"`	Remove trailing spaces
`replace()`	`"a-b-c".replace("-","+")`	`"a+b+c"`	Replace old with new
`find()`	`"hello".find("e")`	1	Return index of first match
`count()`	`"banana".count("a")`	3	Count occurrences
`split()`	`"a,b,c".split(",")`	`["a","b","c"]`	Split into list
`join()`	`"-".join(["a","b","c"])`	`"a-b-c"`	Join list into string

⑥ String Formatting

Insert values inside strings.

Type	Example	Result
% Formatting	`"Name: %s" % "Alice"`	`"Name: Alice"`
`str.format()`	`"Name: {}".format("Alice")`	`"Name: Alice"`
f-Strings	`f"Name: {"Alice"}"`	`"Name: Alice"`

⑦ f-Strings (Python 3.6+)

Clean and readable way to format strings.

```
name = "Alice"
age = 25
msg = f"Hi {name}, you are {age} years old."
print(msg)    # Hi Alice, you are 25 years old.
```

Real-world Use Case

Strings are widely used in:

- ✓ Validating user input
- ✓ Parsing logs and reports
- ✓ Generating emails and templates
- ✓ Building file paths
- ✓ Working with APIs and JSON data

Common Mistake ✗

Strings are immutable.
You cannot change a character like this:

```
s = "hello"
s[0] = 'H'    # ✗ Error!
```

Correct way: `s = "Hello"`

Interview Tip

Frequently asked topics:

- ✓ Difference between `==` and `is`
- ✓ How slicing works with negative index
- ✓ String immutability
- ✓ Use of f-strings vs format()
- ✓ Common string methods

Python Collections

Collections are built-in data structures used to store multiple items in a single variable.

① List

- Ordered
- Mutable
- Allows Duplicates
- Indexing & Slicing

Example:

```
nums = [10, 20, 30, 40]
```

0	1	2	3
10	20	30	40

② Tuple

- Ordered
- Immutable
- Allows Duplicates
- Indexing & Slicing

Example:

```
point = (10, 20, 30)
```

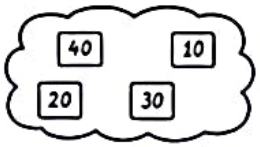
0	1	2
10	20	30

③ Set

- Unordered
- Mutable
- No Duplicates
- No Indexing

Example:

```
unique = {10, 20, 30, 40}
```



④ Dictionary

- Key-Value Pairs
- Mutable
- No Duplicate Keys
- Keys are Unique & Immutable

Example:

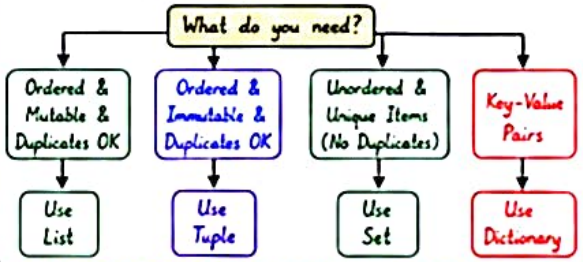
```
user = {'name': 'Alice',
        'age': 25 }
```

Key	Value
'name'	'Alice'
'age'	25

⑤ Common Collection Methods

Method	List	Tuple	Set	Dictionary	Description
append(x)	✓	✗	✗	✗	Adds item x to the end (List only)
extend(iterable)	✓	✗	✗	✗	Adds all items from iterable (List only)
insert(i, x)	✓	✗	✗	✗	Inserts x at index i (List only)
remove(x)	✓	✗	✓	✗	Removes first occurrence of x
pop([i])	✓	✗	✗	✗	Removes & returns item at index i (List only)
clear()	✓	✗	✓	✓	Removes all items
index(x)	✓	✓	✗	✗	Returns index of first occurrence of x
count(x)	✓	✓	✗	✗	Returns count of x
add(x)	✗	✗	✓	✗	Adds x to set
update(iterable)	✗	✗	✓	✗	Adds all items from iterable to set
get(key, default)	✗	✗	✗	✓	Returns value for key, else default
keys() / values() / items()	✗	✗	✗	✓	Returns keys / values / key-value pairs

⑥ Choosing the Right Collection



Pro Tip

- ✓ Use List for dynamic data that changes frequently.
- ✓ Use Tuple for fixed data and as dictionary keys.
- ✓ Use Set for membership testing and removing duplicates.
- ✓ Use Dictionary for lookups and structured data.

Real-World Use Cases

- List : To-do lists, storing records, queues, stacks.
- Tuple : Coordinates, configuration, returning multiple values.
- Set : Removing duplicates, membership testing.
- Dict : JSON data, user profiles, caching, lookups.

Remember

- Lists and Dictionaries are mutable.
- Tuples are immutable.
- Sets are unordered and do not allow duplicates.
- Dictionary keys must be immutable (string, number, tuple etc.).

Comprehensions & Lambda

Comprehensions provide a concise way to create collections.
Lambda creates small anonymous functions.

① List Comprehension

Creates a new list by applying an expression for each item in an iterable.

Syntax:
```
[expression for item in iterable if condition]
```

Use Case: Transform or filter data in lists.

```
# Squares of first 5 numbers
squares = [x*x for x in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]
```

② Dictionary Comprehension

Creates a new dictionary by iterating over an iterable and generating key-value pairs.

Syntax:
```
{key_expr: value_expr for item in iterable if condition}
```

Use Case: Build dictionaries from existing data.

```
# Number : Square mapping
squares = {x: x*x for x in range(1, 6)}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

{ key : value }

③ Set Comprehension

Creates a new set by applying an expression for each item in an iterable.

Syntax:
```
{expression for item in iterable if condition}
```

Use Case: Get unique values or remove duplicates.

```
# Unique even numbers from 1 to 10
evens = {x for x in range(1, 11) if x % 2 == 0}
print(evens)  # {2, 4, 6, 8, 10}
```

④ Lambda Functions

Small anonymous functions defined with the lambda keyword.

Syntax:
```
lambda arguments: expression
```

```
add = lambda a, b: a + b
print(add(5, 3))   # 8
square = lambda x: x * x
print(square(4))   # 16
```

Use Case: Short functions, used with higher-order functions (like map(), filter(), etc.

λ

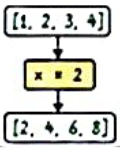
⑤ map() Function

Applies a function to all items in an iterable and returns a map object.

Syntax:
```
map(function, iterable)
```

Use Case: Transform every item in an iterable

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x*2, nums))
print(doubled)  # [2, 4, 6, 8]
```



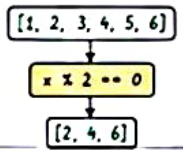
⑥ filter() Function

Filters elements from an iterable using a function that returns True or False.

Syntax:
```
filter(function, iterable)
```

Use Case: Filter data based on a condition.

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)  # [2, 4, 6]
```



⑦ Quick Comparison

Feature	List Comp	Dict Comp	Set Comp	map()	filter()	lambda
Syntax	[expr for item in iter if cond]	{k: v for item in iter if cond}	{expr for item in iter if cond}	map(func, iterable)	filter(func, iterable)	lambda args: expr
Returns	List	Dictionary	Set	map object (convertible to list)	filter object (convertible to list)	Function
Use	Transform & filter into a list	Create dicts from data	Unique values from data	Transform each item	Select items that match condition	Create small anonymous functions
Example	[x*2 for x in nums]	{x: x*2 for x in nums}	{x for x in nums if x%2==0}	map(lambda x: x*2, nums)	filter(lambda x: x%2==0, nums)	lambda x: x*2

Real-world Use Cases

- ✓ List Comp → Data transformation, report building
- ✓ Dict Comp → Create lookup maps, config data
- ✓ Set Comp → Remove duplicates, uniqueness checks
- ✓ map() → Apply calculations, format conversions
- ✓ filter() → Data validation, filtering records
- ✓ lambda → Callbacks, short reusable logic

Interview Tip

- ✓ Comprehensions are more readable and faster than traditional loops
- ✓ Use set for uniqueness
- ✓ map() and filter() return iterators in Python 3. Convert to list when needed
- ✓ Lambda is restricted to a single expression

Common Mistakes

- ✗ Forgetting to convert map/filter objects to list for printing
- ✗ Using list when set is required
- ✗ Overusing lambda (hurts readability).
- ✗ Not handling conditions properly in comprehensions.

Modules & Packages in Python

Modules are Python files (.py) that contain code.
Packages are collections of modules in a directory.

① Modules

A module is a single Python file that contains functions, classes and variables.

```python
# mymodule.py
PI = 3.14159
def add(a, b):
    return a + b
class Calculator:
    def mul(self, a, b):
        return a * b
```

Use Case:
Organize reusable code in separate files for better maintainability.

② Packages

A package is a directory containing multiple modules and an optional __init__.py files.

```
mypackage/
    __init__.py
    math_ops.py
    string_ops.py
```

Use Case:
Structure large projects logically by grouping related modules.

⭐ __init__.py makes Python treat the directory as a package.

③ Import

Import entire module and access members.

```python
import mymodule
print(mymodule.PI)
result = mymodule.add(10, 20)
calc = mymodule.Calculator()
print(calc.mul(3, 4))
```

Output:

```
3.14159
30
12
```

④ from ... import

Import specific members from a module.

```python
from mymodule import add, PI
print(PI)
print(add(5, 7))
```

Output:

```
3.14159
12
```

⑤ Built-in Modules

Python comes with many built-in modules.
No installation required

Module	Purpose	Example
math	Mathematical functions	math.sqrt(16)
random	Random number generation	random.randint(1, 10)
datetime	Date & time operations	datetime.now()
os	Operating system interface	os.listdir(".")
sys	System-specific parameters	sys.version

⑥ pip (Python Package Manager)

Used to install third-party packages from Python Package Index (PyPI).

- Install a package
  `pip install requests`
- Install specific version
  `pip install pandas==2.1.4`
- list installed packages
  `pip list`
- Uninstall a package
  `pip uninstall requests`

Common Packages:

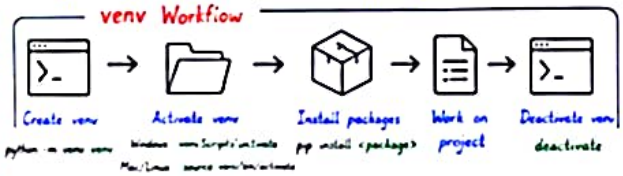
- requests → HTTP requests
- numpy → Numerical computing
- pandas → Data analysis
- matplotlib → Data visualization
- pytest → Testing framework

⑦ Virtual Environment (venv)

Creates an isolated environment for your Python project with its own packages.

Why use venv?
- ✓ Keeps project dependencies isolated
- ✓ Avoids version conflicts
- ✓ Makes project more portable
- ✓ Cleaner & professional workflow

venv Workflow



Project Structure

```
my_project/
    venv/
        Scripts/ (or bin/)
        Lib/ (or lib/)
    main.py
    requirements.txt
    README.md
```

⭐ Tip: Always add venv/ to .gitignore to avoid pushing environment files to Git.

Interview Tip

- ✓ Module: single .py file.
- ✓ Package: directory of modules.
- ✓ __init__.py identifies a package
- ✓ Use venv + requirements.txt in real projects

Common Mistakes

- ✗ Forgetting to activate venv
- ✗ Installing packages globally
- ✗ Pushing venv folder to Git
- ✗ Confusing module name with package name

Real-world Use Cases

- ✓ requests → Call REST APIs in automation
- ✓ pandas → Read CSV/Excel test data
- ✓ pytest → Run automated test suites
- ✓ logging, datetime → Generate reports with timestamps

Python File Handling

File handling allows Python programs to read from and write to files stored on disk.

① Open a File

Use open() to open a file.

Syntax:

```
open(file, mode, encoding)
```

Common Modes:

'r'	Read (default)
'w'	Write (overwrites)
'a'	Append (adds to end)
'x'	Create (fails if exists)
'b'	Binary mode
't'	Text mode (default)

✓ Always close files!

② Read Files

Read entire file or line by line.

```
# Read entire file
with open('data.txt', 'r',
    encoding='utf-8') as f:
    content = f.read()
    print(content)

# Read line by line
with open('data.txt','r') as f:
    for line in f:
        print(line.strip())
```

③ Write Files

Write data to a file.

```
with open('output.txt', 'w',
    encoding='utf-8') as f:

f.write('Python is awesome!')
```

Note:
Mode 'w' overwrites the file if it already exists.

④ Append Files

Add content to the end of an existing file.

```
with open('log.txt', 'a',
    encoding='utf-8') as f:
f.write('New log entry\n')
f.write('Another entry\n')
```

Note:
Mode 'a' does not overwrite. It appends new data.

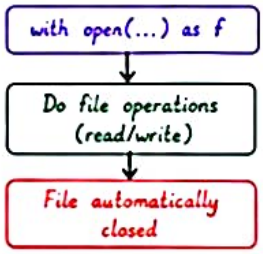
⑤ with Statement (Best Practice)

```
with open('data.txt', 'r') as f:
    data = f.read()
```

✓ File is automatically closed after block ends even if error occurs.

Benefits:

- ✓ Automatically closes the file
- ✓ Prevents resource leaks
- ✓ Cleaner and safer code



⑥ CSV Files

Work with CSV files using csv module.

```
import csv
with open('data.csv', newline='',
    encoding='utf-8') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

Write CSV:

```
with open('out.csv', 'w', newline='',
    encoding='utf-8') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(['Name', 'Age'])
    writer.writerow(['Alice', 25])
```

✓ Great for tabular data like reports.

⑦ JSON Files

Work with JSON files using json module.

```
import json
# Read JSON
with open('data.json', 'r',
    encoding='utf-8') as f:
    data = json.load(f)
    print(data)
# Write JSON
info = {'name': 'Alice', 'age': 25}
with open('info.json', 'w',
    encoding='utf-8') as f:
    json.dump(info, f, indent=4)
```

✓ Ideal for APIs, configs, and structured data.

⑧ Quick Summary

- ✓ Use open() to open files.
- ✓ Use modes 'r', 'w', 'a', 'x'.
- ✓ Use with statement.
- ✓ Use read(), readline(), readlines() to read.
- ✓ Use write() to write.
- ✓ Use csv module for CSV files.
- ✓ Use json module for JSON files.

Pro Tip
Always handle exceptions while working with files. Use try-except for robust code.

Exception Handling in Python

Exception handling helps us handle runtime errors gracefully so that our program doesn't crash.

① Try – Except – Else – Finally

Handle errors and define cleanup actions.

```
try:
    x = 10 / int(input("Enter a number: "))
    print("Result:", x)
except ValueError:
    print("Invalid input! Please enter a number:")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Execution successful.")
finally:
    print("Cleanup: This will run always.")
```

- `try` → Code that might raise an exception
- `except ValueError` → Handles specific exceptions
- `else` → Runs if no exception occurs
- `finally` → Runs always (cleanup code)

② Raise Keyword

Manually raise an exception.

```
age = int(input("Enter age: "))
if age < 18:
    raise ValueError("Age must be 18 or above")
print("Access granted")
```

⭐ Use `raise` to enforce business rules and validate conditions.

③ Custom Exceptions

Create your own exception classes.

```
class InvalidAgeError(Exception):
    """Raised when age is not valid."""
    pass

age = int(input("Enter age:"))
if age < 0:
    raise InvalidAgeError("Age cannot be negative")
```

Inherit from `Exception` to create custom errors.

④ Common Built-in Exceptions

Exception	When it Occurs
ValueError	Invalid value/argument
ZeroDivisionError	Division by zero
TypeError	Operation on wrong type
FileNotFoundError	File not found
KeyError	Dictionary key not found
IndexError	Index out of range

⑤ Debugging Tips

- ✓ Read the full error message carefully.
- ✓ Check the line number mentioned in the traceback.
- ✓ Print variable values using `print()` before the error line.
- ✓ Use `try...except` to isolate the error.
- ✓ Use debugger tools (breakpoints, step into, step over).
- ✓ Use `logging` instead of `print()` in large projects.
- ✓ Keep code modular to make debugging easier.

Pro Tip

Handle specific exceptions instead of using a generic `except Exception`. It helps in debugging and makes code reliable.

⑥ Traceback Example

Error traceback shows the call stack.

```
Traceback (most recent call last):
  File "app.py", line 4, in <module>
    x = 10 / 0
ZeroDivisionError: division by zero
```

Read from bottom to top to find the actual error location.

⑦ Best Practices

- ✓ Handle specific exceptions.
- ✓ Don't ignore exceptions silently.
- ✓ Keep `finally` block for resource cleanup.
- ✓ Log exceptions for future analysis.
- ✓ Be user-friendly in error messages.
- ✓ Avoid using bare `except`.
- ✓ Validate inputs and fail early.

⭐ Real-world Use Case

Exception handling is used in APIs, file processing, automation scripts, data pipelines, and production systems to prevent unexpected crashes.

Object-Oriented Programming in Python

OOP helps us model real-world entities using classes and objects.
It improves code reusability, maintainability and scalability.

① Classes

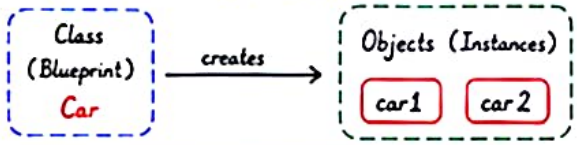
A class is a blueprint (template) for creating objects

```
class Car:
    brand = "Toyota"   # class variable
    def start(self):   # method
        print("Car started")
```

Key Point: Class defines attributes (data) and methods (behavior).

② Objects

An object is an instance of a class.




```
car1 = Car()
car2 = Car()
car1.brand      # Access class variable
car1.start()    # Call method
```

Tip: Each object has its own state and behavior.

③ Constructors (__init__)

A constructor is called automatically when an object is created.

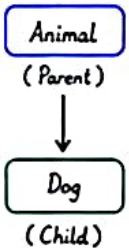
```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def info(self):
        print(f"{self.name} is {self.age} years old")

p = Person("Alice", 25)
p.info()  # Alice is 25 years old
```

Note: __init__ is called a constructor.

④ Inheritance

A child class inherits attributes and methods from a parent class.

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal):
    def speak(self):
        print("Dog barks")
```



⑤ Polymorphism (Many Forms)

Same method name, different behavior.

```
def speak(obj):
    obj.speak()
```

Animal() → speak() → Animal speaks
Dog() → speak() → Dog barks

Tip: Polymorphism provides flexibility in code.

⑥ Encapsulation (Data Hiding)

Bundle data and methods together and restrict direct access to data

```
class Account:
    def __init__(self, balance):
        self.__balance = balance  # private
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
    def get_balance(self):
        return self.__balance
```

Note: Double underscore makes attribute name-mangled (harder to access directly).

⑦ Abstraction (Hiding Complexity)

Show only essential features and hide implementation details

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Shape (Abstract) → area()
Circle (Concrete) → area()

Note: Abstract classes cannot be instantiated. They define a blueprint for subclasses.

Quick OOP Summary

- ✓ Class → Blueprint
- ✓ Object → Instance of class
- ✓ Constructor → __init__()
- ✓ Inheritance → Reuse code
- ✓ Polymorphism → Many forms
- ✓ Encapsulation → Protect data
- ✓ Abstraction → Hide complexity

Interview Tip

Focus on the "why" behind each concept with examples.

Common Mistakes

- ✗ Forgetting __init__ in class
- ✗ Overusing inheritance
- ✗ Not using encapsulation
- ✗ Not implementing abstract methods
- ✗ Confusing abstraction with encapsulation

Real-world Use Cases

- ✓ Frameworks (Django, Flask) → OOP heavily used
- ✓ Selenium Page Object Model → Classes & Objects
- ✓ Banking systems → Encapsulation
- ✓ APIs & SDKs → Abstraction & Polymorphism

Advanced Python

Level up your Python skills with powerful advanced features.

① Iterators

An iterator returns one item at a time from a collection.

Syntax: `iter(obj)` → iterator with `__next__()`

```
nums = [1, 2, 3]
it = iter(nums)
print(next(it))   # 1
print(next(it))   # 2
```

Use Case: Memory efficient way to process large data.

② Generators

Generate values on the fly using `yield` (lazy evaluation).

Syntax:
```
def func():
    yield value
```

```
def count(n):
    for i in range(n):
        yield i

for num in count(3):
    print(num)   # 0 1 2
```

Advantages:
- ✓ Low memory usage
- ✓ Faster for large data
- ✓ Produces values one at a time

③ Decorators

Add extra behavior to functions without modifying them.

Syntax:
```
@decorator_name
def function():
```

```
def log_time(func):
    def wrapper(*args, **kwargs):
        print("Function started...")
        result = func(*args, **kwargs)
        print("Function ended...")
        return result
    return wrapper
```

Use Case: Logging, auth, retry mechanism in automation scripts.

④ Context Managers

Manage resources and ensure proper cleanup.

Syntax:
```
with expression as variable:
    # code block
```

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Benefits:
- ✓ Auto resource cleanup
- ✓ No need to call close() manually
- ✓ Works even if exception occurs

⑤ Useful Built-in Functions

Powerful functions for everyday Python programming.

Function	Purpose
len()	Returns length of a collection
sorted()	Returns sorted list
enumerate()	Returns index + value
zip()	Combines multiple iterables
any()	True if any element is True
all()	True if all elements are True
isinstance()	Checks object type

⑥ Performance Tips

Write faster, cleaner and memory efficient Python code.

- ✓ Use list / dict comprehensions (faster than loops)
- ✓ Prefer generators for large data
- ✓ Use built-in functions (they are optimized)
- ✓ Avoid global variables
- ✓ Use sets for fast membership tests (O(1))
- ✓ Profile code using timeit or cProfile
- ✓ Use local variables inside loops

Interview Tip

- ✓ Know the difference: Iterator vs Generator
- ✓ Understand real-world use cases
- ✓ Decorators and Context Managers are frequently asked in interviews

Key Takeaways

- ✓ Iterators → iterate efficiently
- ✓ Generators → save memory
- ✓ Decorators → extend functionality
- ✓ Context Managers → handle resources
- ✓ Built-ins → write cleaner code

Real-world Use Cases

- ✓ API response streaming (Generators)
- ✓ Logging & retry (Decorators)
- ✓ File / DB connections (Context Managers)
- ✓ Data processing & automation scripts
- ✓ Performance critical applications